Lab5: Computer Architectuur

Stijn Verhaeghe & Andres Bosman

March 2026

1 Introductie

In dit labo is het de bedoeling om te kijken wat de invloed is van synchroon en asynchroon operatie. Bij een synchroon operatie roept de CPU de kernel op, ondertussen wacht de CPU tot de kernel gedaan is. Bij de asynchroon operatie voert de CPU ook al operaties uit terwijl de GPU de kernel uitvoert. We verwachten dat de asynchroon operatie rapper is dan de synchroon operatie omdat je tijd wint door de arrays klaar te zetten voor de volgende bewerking wanneer de GPU zijn bewerkingen uitvoert.

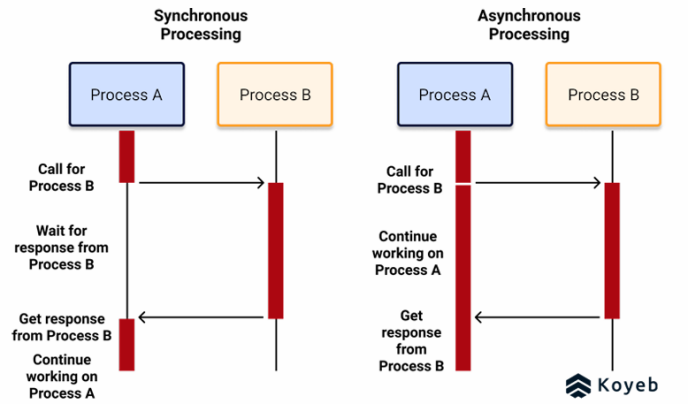


Figure 1: Verschil synchroon en asynchroon processing

2 Oefening

Het labo bestaat uit één oefening waar het verschil tussen synchroon en asynchroon processing onderzocht wordt. De bedoeling is om een kernel te maken die in totaal vier operaties kan uitvoeren: sum, prod, min en max. Deze vier operaties worden na elkaar uitgevoerd een keer synchroon en een keer asynchroon. De verwachting is dat de asynchrone werking sneller is dan de synchrone werking.

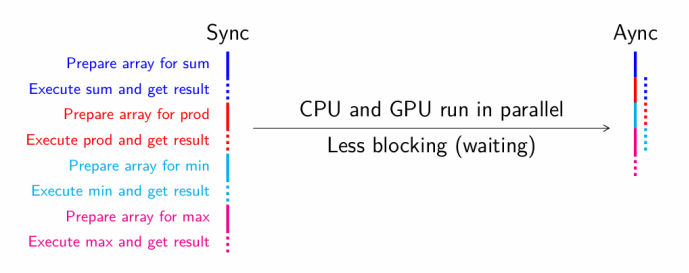


Figure 2: Oefening in figuur vorm

2.1 Resultaten

We verwachten dat de asynchrone versie sneller loopt dan de synchrone versie. Maar de resultaten tonen het tegenovergestelde. De synchrone versie is bij de array van 3 elementen 0,5ms sneller dan de asynchrone versie. Naarmate het aantal elementen toeneemt neemt het verschil af, Vanaf een kwart miljoen elementen wordt asynchroon sneller dan synchroon (3ms sneller).
Dit komt omdat voor kleine arrays de GPU te maken krijgt met een relatief hoge overhead voor het opstarten van kernels en de datatransfer tussen de geheugens, waardoor de winst in rekensnelheid pas opweegt tegen deze kosten bij grotere hoeveelheden data.

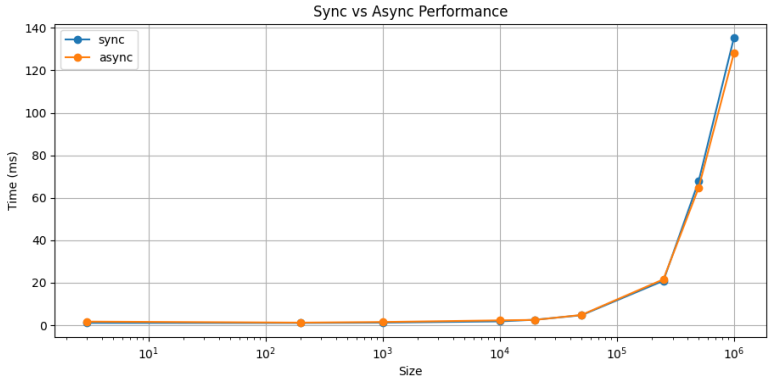


Figure 3: asynchroon vs synchroon

3 Conclusie

Asynchrone kan sneller zijn bij heel grootte arrays, je moet waarschijnlijk wel rekening houden met welke functies je gebruikt en hoe je zo oproept. Hoe wij het deden zou ervoor kunnen zorgen dat de code niet volledig asynchroon loopt. (bij de min/max)

4 Code

4.1 async

```
#include <stdio.h>
#include <cuda_runtime.h>
#include <stdlib.h>
#include <time.h>
#include <limits.h>
#include <vector>
#include <iostream>
#include <iomanip>

__global__ void findMaxReductionKernel(int* d_in, int* d_out, int n) {
    int tid = threadIdx.x;
    int i = blockIdx.x * blockDim.x + threadIdx.x;

    // Elke pass halveert het aantal actieve threads in een block
    for (unsigned int s = blockDim.x / 2; s > 0; s >>= 1) {
        if (tid < s && (i + s) < n) {
            if (d_in[i + s] > d_in[i]) {
                d_in[i] = d_in[i + s];
            }
        }
        __syncthreads();
    }

    // De winnaar van dit block schrijft naar de output array
    if (tid == 0) {
        d_out[blockIdx.x] = d_in[i];
    }
}

__global__ void findMinReductionKernel(int* d_in, int* d_out, int n) {
    int tid = threadIdx.x;
    int i = blockIdx.x * blockDim.x + threadIdx.x;

    // Elke pass halveert het aantal actieve threads in een block
    for (unsigned int s = blockDim.x / 2; s > 0; s >>= 1) {
        if (tid < s && (i + s) < n) {
            if (d_in[i + s] < d_in[i]) {
                d_in[i] = d_in[i + s];
            }
        }
```

```
41          __syncthreads();
42      }
43
44      // De winnaar van dit block schrijft naar de output
            array
45      if (tid == 0) {
46          d_out[blockIdx.x] = d_in[i];
47      }
48  }
49
50  __global__ void findMinAtomic(int* d_array, int* d_max, int
        n) {
51      int idx = blockIdx.x * blockDim.x + threadIdx.x;
52      if (idx < n) atomicMin(d_max, d_array[idx]);
53  }
54
55
56  // Pass 1: each block reduces its chunk into partialProducts
        []
57  __global__ void reduceMultiplyFloat(float *input, float *
        partialProducts, int size)
58  {
59      extern __shared__ float sdataFloat[];
60
61      int tid = threadIdx.x;
62      int idx = blockIdx.x * blockDim.x + tid;
63
64      sdataFloat[tid] = (idx < size) ? input[idx] : 1.0f;
65      __syncthreads();
66
67      for (int s = blockDim.x / 2; s > 0; s >>= 1) {
68          if (tid < s) {
69              sdataFloat[tid] *= sdataFloat[tid + s];
70          }
71          __syncthreads();
72      }
73
74      if (tid == 0) {
75          partialProducts[blockIdx.x] = sdataFloat[0];
76      }
77  }
78
79  __global__ void reduceMultiplyFinalFloat(float *
        partialProducts, float *result, int size)
80  {
81      extern __shared__ float sdataFloat[];
82
83      int tid = threadIdx.x;
84      sdataFloat[tid] = (tid < size) ? partialProducts[tid] :
            1.0f;
```

```
85      __syncthreads();
86
87      for (int s = blockDim.x / 2; s > 0; s >>= 1) {
88          if (tid < s) {
89              sdataFloat[tid] *= sdataFloat[tid + s];
90          }
91          __syncthreads();
92      }
93
94      if (tid == 0) {
95          *result = sdataFloat[0];
96      }
97  }
98  __global__ void reduceSum(int *input, int *partialSums, int
        size)
99  {
100     extern __shared__ int sdata[];
101
102     int tid = threadIdx.x;
103     int idx = blockIdx.x * blockDim.x + tid;
104
105     sdata[tid] = (idx < size) ? input[idx] : 0;  // identity
            for sum is 0
106     __syncthreads();
107
108     for (int s = blockDim.x / 2; s > 0; s >>= 1) {
109         if (tid < s) {
110             sdata[tid] += sdata[tid + s];  // + instead of *
111         }
112         __syncthreads();
113     }
114
115     if (tid == 0) {
116         partialSums[blockIdx.x] = sdata[0];
117     }
118 }
119
120 __global__ void reduceSumFinal(int *partialSums, int *result
        , int size)
121 {
122     extern __shared__ int sdata[];
123
124     int tid = threadIdx.x;
125     sdata[tid] = (tid < size) ? partialSums[tid] : 0;  //
            identity for sum is 0
126     __syncthreads();
127
128     for (int s = blockDim.x / 2; s > 0; s >>= 1) {
129         if (tid < s) {
130             sdata[tid] += sdata[tid + s];
```

```
131         }
132         __syncthreads();
133     }
134 
135     if (tid == 0) {
136         *result = sdata[0];
137     }
138 }
139 
140 void printMatrix(const std::vector<int>& m, int rows, int
    collumns) {
141     for(int r = 0; r < rows; r++) {
142         for(int c = 0; c < collumns; c++) {
143             std::cout << m[r * collumns + c] << "␣";
144         }
145         std::cout << std::endl;
146     }
147 }
148 
149 void printArray(const std::vector<int>& m, int size) {
150         std:: cout << "[";
151     for(int r = 0; r < size; r++) {
152         if(r!=size-1){
153             std::cout << m[r] << ",␣";
154         }
155         else{std::cout << m[r];}
156     }
157             std::cout << "]"<< std::endl;
158 
159 }
160 
161 void printArrayFloat(const std::vector<float>& m, int size)
    {
162         std:: cout << "[";
163     for(int r = 0; r < size; r++) {
164         if(r!=size-1){
165             std::cout << m[r] << ",␣";
166         }
167         else{std::cout << m[r];}
168     }
169             std::cout << "]"<< std::endl;
170 
171 }
172 void printResult(const std::vector<int>& m1, const std::
    vector<int>& m2, const std::vector<int>& result_matrix,
    int rows, int collumns){
173         std::cout << std::string(20, '=') << std::endl<<std
            ::endl;
174         printMatrix(m1, rows, collumns);
```

```
175         std::cout<<std::endl << "␣plus␣min␣maal,␣denk␣na␣en␣
                je␣zult␣het␣wel␣zien␣" << std::endl<<std::endl;
176         printMatrix(m2, rows, collumns);
177         std::cout << std::string(20, '-') << std::endl;
178         printMatrix(result_matrix, rows, collumns);
179         std::cout << std::string(20, '=') << std::endl<<std
                ::endl;
180 }
181
182
183 std::vector<int> generateArray(int size) {
184     std::vector<int> array(size);
185     for(int i = 0; i < size; i++)
186         array[i] = rand() % 3;  //rest van deling door 10 ->
                0-9, veilig voor mult (als size<max_int)
187     return array;
188 }
189
190 std::vector<float> generateArrayfloat(int size) {
191     std::vector<float> array(size);
192     for(int i = 0; i < size; i++)
193         array[i] = static_cast<float>(rand()) / static_cast<
                float>(RAND_MAX); // [0.0, 1.0]
194     return array;
195 }
196
197 int main() {
198     std::vector<int> sizes = {50000, 250000, 500000,
            1000000}; // Test size
199     srand(42);
200     float ms;
201
202
203     int output;
204     int threads1D = 256;
205
206     int* d_a1, *d_a3, *d_a4;
207     float* d_a2;
208     int* d_result_matrix;
209     int* d_out;
210     int *d_partialSum, *d_sumResult;
211     float *d_partial;
212
213     cudaEvent_t start_cuda, stop_cuda;
214     cudaEventCreate(&start_cuda);
215     cudaEventCreate(&stop_cuda);
216     cudaStream_t stream1, stream2, stream3, stream4;
217     cudaStreamCreate(&stream1);
218     cudaStreamCreate(&stream2);
219     cudaStreamCreate(&stream3);
```

```
220     cudaStreamCreate(&stream4);
221
222     std::cout << "␣asynchrone:␣" << std::endl << std::setw
            (20) << "size" << "␣" << "␣time"<<std::endl;
223     for(int size : sizes){
224         //volledige lus in rekening
225         cudaEventRecord(start_cuda);
226
227         for(int iterations = 0; iterations < 50; iterations
                ++){
228         //max mult
229         // long max_mult = 1;
230         // for(int i = 0; i<size; i++){
231         //     max_mult *= 3;
232         // }
233         // if(INT_MAX <= max_mult | max_mult < 0){
234         //     std::cout<<"max mult bij size: " << size << "
                    kan integer overflow geven"<<std::endl;
235         // }
236         // std::cout << std::string(90, '-') << std::endl;
237
238         int rows = size;
239         int columns = size;
240         int total_elements = size;
241
242         int blocks1D = (total_elements + threads1D - 1) /
                threads1D;
243         float *d_result;
244         // Allocate device memory once
245         cudaMalloc(&d_a1, total_elements * sizeof(int));
246         cudaMalloc(&d_a2, total_elements * sizeof(float));
247         cudaMalloc(&d_a3, total_elements * sizeof(int));
248         cudaMalloc(&d_a4, total_elements * sizeof(int));
249         cudaMalloc(&d_result_matrix, total_elements * sizeof
                (int));
250         cudaMalloc(&d_out, ((total_elements + threads1D - 1)
                / threads1D) * sizeof(int));
251         cudaMalloc(&d_partial, blocks1D * sizeof(int));
252         cudaMalloc(&d_result, sizeof(float));
253         cudaMalloc(&d_partialSum, blocks1D * sizeof(int));
254         cudaMalloc(&d_sumResult, sizeof(int));
255
256         // Step 1: Generate m1 and copy async
257         std::vector<int> m1 = generateArray(size);
258         cudaMemcpyAsync(d_a1, m1.data(), total_elements *
                sizeof(int),
259                         cudaMemcpyHostToDevice, stream1);
260
261         // Step 2: While m1 copy is running, generate m2
262         std::vector<int> m2 = generateArray(size);
```

```
263         cudaMemcpyAsync(d_a2, m2.data(), total_elements *
                sizeof(float),
264                         cudaMemcpyHostToDevice, stream2);
265
266         // Step 3: Generate m3 and m4 in parallel while GPU
                copies m1/m2
267         std::vector<int> m3 = generateArray(size);
268         std::vector<int> m4 = generateArray(size);
269         cudaMemcpyAsync(d_a3, m3.data(), total_elements *
                sizeof(int),
270                         cudaMemcpyHostToDevice, stream3);
271         cudaMemcpyAsync(d_a4, m4.data(), total_elements *
                sizeof(int),
272                         cudaMemcpyHostToDevice, stream4);
273
274         // ---------------------------
275         // SUM
276         reduceSum<<<blocks1D, threads1D, threads1D*sizeof(
                int), stream1>>>(d_a1, d_partialSum, size);
277         int finalThreads = 1;
278         while(finalThreads < blocks1D) finalThreads <<= 1;
279         reduceSumFinal<<<1, finalThreads, finalThreads*
                sizeof(int), stream1>>>(d_partialSum, d_sumResult
                , blocks1D);
280
281         int sumOutput;
282         cudaMemcpyAsync(&sumOutput, d_sumResult, sizeof(int)
                , cudaMemcpyDeviceToHost, stream1);
283
284         // ---------------------------
285         // MULTIPLICATION
286         reduceMultiplyFloat<<<blocks1D, threads1D, threads1D
                *sizeof(float), stream2>>>(d_a2, d_partial,
                total_elements);        finalThreads = 1;
287         while(finalThreads < blocks1D) finalThreads <<= 1;
288         reduceMultiplyFinalFloat<<<1, finalThreads,
                finalThreads*sizeof(float), stream2>>>(d_partial,
                 d_result, blocks1D);
289         int multOutput;
290         cudaMemcpyAsync(&multOutput, d_result, sizeof(int),
                cudaMemcpyDeviceToHost, stream2);
291
292         // ---------------------------
293         // MAX
294         int max_size = total_elements;
295         int* d_in = d_a3;
296         int* d_tmp = d_out;
297         while(max_size > 1){
298             int blocksPerGrid = (max_size + threads1D - 1) /
                    threads1D;
```

```
299             findMaxReductionKernel<<<blocksPerGrid,
                    threads1D, 0, stream3>>>(d_in, d_tmp,
                    max_size);
300             cudaStreamSynchronize(stream3);
301             int* temp = d_in; d_in = d_tmp; d_tmp = temp;
302             max_size = blocksPerGrid;
303         }
304         int maxOutput;
305         cudaMemcpyAsync(&maxOutput, d_in, sizeof(int),
                cudaMemcpyDeviceToHost, stream3);
306
307         // --------------------------
308         // MIN
309         max_size = total_elements;
310         d_in = d_a4;
311         d_tmp = d_out;
312         while(max_size > 1){
313             int blocksPerGrid = (max_size + threads1D - 1) /
                     threads1D;
314             findMinReductionKernel<<<blocksPerGrid,
                    threads1D, 0, stream4>>>(d_in, d_tmp,
                    max_size);
315             cudaStreamSynchronize(stream4);
316             int* temp = d_in; d_in = d_tmp; d_tmp = temp;
317             max_size = blocksPerGrid;
318         }
319         int minOutput;
320         cudaMemcpyAsync(&minOutput, d_in, sizeof(int),
                cudaMemcpyDeviceToHost, stream4);
321
322         // --------------------------
323         // Wait for all streams to finish
324         cudaStreamSynchronize(stream1);
325         cudaStreamSynchronize(stream2);
326         cudaStreamSynchronize(stream3);
327         cudaStreamSynchronize(stream4);
328
329         // Print results
330         //std::cout << "Array 1: "; printArray(m1, size);
                std::cout << "Total SUM: " << sumOutput << std::
                endl;
331         //std::cout << "Array 2: "; printArray(m2, size);
                std::cout << "Total MULT: " << multOutput << std
                ::endl;
332         //std::cout << "Array 3: "; printArray(m3, size);
                std::cout << "MAX: " << maxOutput << std::endl;
333         //std::cout << "Array 4: "; printArray(m4, size);
                std::cout << "MIN: " << minOutput << std::endl;
334
335         // Free device memory
```

```
336         cudaFree(d_a1); cudaFree(d_a2); cudaFree(d_a3);
                cudaFree(d_a4);
337         cudaFree(d_result_matrix); cudaFree(d_out);
338         cudaFree(d_partial); cudaFree(d_result);
339         cudaFree(d_partialSum); cudaFree(d_sumResult);
340   }
341
342         //volledige lus in rekening brengen...
343         cudaEventRecord(stop_cuda);
344         cudaEventSynchronize(stop_cuda);
345         cudaEventElapsedTime(&ms, start_cuda, stop_cuda);
346         float avgTime = ms/50;
347         std::cout << std::setw(20) << size << "␣" << avgTime
                <<std::endl;
348
349     }
350
351     // Destroy streams
352     cudaStreamDestroy(stream1);
353     cudaStreamDestroy(stream2);
354     cudaStreamDestroy(stream3);
355     cudaStreamDestroy(stream4);
356
357     return 0;
358 }
```

4.2 sync

```
1  #include <stdio.h>
2  #include <cuda_runtime.h>
3  #include <stdlib.h>
4  #include <time.h>
5  #include <limits.h>
6  #include <vector>
7  #include <iostream>
8  #include <iomanip>
9
10 __global__ void findMaxReductionKernel(int* d_in, int* d_out
       , int n) {
11     int tid = threadIdx.x;
12     int i = blockIdx.x * blockDim.x + threadIdx.x;
13
14     // Elke pass halveert het aantal actieve threads in een
           block
15     for (unsigned int s = blockDim.x / 2; s > 0; s >>= 1) {
16         if (tid < s && (i + s) < n) {
17             if (d_in[i + s] > d_in[i]) {
18                 d_in[i] = d_in[i + s];
```

```
            }
        }
        __syncthreads();
    }

    // De winnaar van dit block schrijft naar de output array
    if (tid == 0) {
        d_out[blockIdx.x] = d_in[i];
    }
}

__global__ void findMinReductionKernel(int* d_in, int* d_out, int n) {
    int tid = threadIdx.x;
    int i = blockIdx.x * blockDim.x + threadIdx.x;

    // Elke pass halveert het aantal actieve threads in een block
    for (unsigned int s = blockDim.x / 2; s > 0; s >>= 1) {
        if (tid < s && (i + s) < n) {
            if (d_in[i + s] < d_in[i]) {
                d_in[i] = d_in[i + s];
            }
        }
        __syncthreads();
    }

    // De winnaar van dit block schrijft naar de output array
    if (tid == 0) {
        d_out[blockIdx.x] = d_in[i];
    }
}

__global__ void findMinAtomic(int* d_array, int* d_max, int n) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < n) atomicMin(d_max, d_array[idx]);
}


// Pass 1: each block reduces its chunk into partialProducts[]
__global__ void reduceMultiply(int *input, int *partialProducts, int size)
{
    extern __shared__ int sdata[];

    int tid = threadIdx.x;
```

```
    int idx = blockIdx.x * blockDim.x + tid;

    // Load into shared memory (use 1 as identity if out of bounds)
    sdata[tid] = (idx < size) ? input[idx] : 1;
    __syncthreads();

    // Reduction in shared memory
    for (int s = blockDim.x / 2; s > 0; s >>= 1) {
        if (tid < s) {
            sdata[tid] *= sdata[tid + s];
        }
        __syncthreads();
    }

    // Write block result
    if (tid == 0) {
        partialProducts[blockIdx.x] = sdata[0];
    }
}

// Pass 2: reduce the partial products (usually small enough for one block)
__global__ void reduceMultiplyFinal(int *partialProducts, int *result, int size)
{
    extern __shared__ int sdata[];

    int tid = threadIdx.x;
    sdata[tid] = (tid < size) ? partialProducts[tid] : 1;
    __syncthreads();

    for (int s = blockDim.x / 2; s > 0; s >>= 1) {
        if (tid < s) {
            sdata[tid] *= sdata[tid + s];
        }
        __syncthreads();
    }

    if (tid == 0) {
        *result = sdata[0];
    }
}

__global__ void reduceMultiplyFloat(float *input, float *partialProducts, int size)
{
    extern __shared__ float sdataFloat[];

    int tid = threadIdx.x;
```

```
108     int idx = blockIdx.x * blockDim.x + tid;
109 
110     sdataFloat[tid] = (idx < size) ? input[idx] : 1.0f;
111     __syncthreads();
112 
113     for (int s = blockDim.x / 2; s > 0; s >>= 1) {
114         if (tid < s) {
115             sdataFloat[tid] *= sdataFloat[tid + s];
116         }
117         __syncthreads();
118     }
119 
120     if (tid == 0) {
121         partialProducts[blockIdx.x] = sdataFloat[0];
122     }
123 }
124 
125 __global__ void reduceMultiplyFinalFloat(float *
        partialProducts, float *result, int size)
126 {
127     extern __shared__ float sdataFloat[];
128 
129     int tid = threadIdx.x;
130     sdataFloat[tid] = (tid < size) ? partialProducts[tid] :
            1.0f;
131     __syncthreads();
132 
133     for (int s = blockDim.x / 2; s > 0; s >>= 1) {
134         if (tid < s) {
135             sdataFloat[tid] *= sdataFloat[tid + s];
136         }
137         __syncthreads();
138     }
139 
140     if (tid == 0) {
141         *result = sdataFloat[0];
142     }
143 }
144 __global__ void reduceSum(int *input, int *partialSums, int
        size)
145 {
146     extern __shared__ int sdata[];
147 
148     int tid = threadIdx.x;
149     int idx = blockIdx.x * blockDim.x + tid;
150 
151     sdata[tid] = (idx < size) ? input[idx] : 0;  // identity
            for sum is 0
152     __syncthreads();
153 
```

```
154     for (int s = blockDim.x / 2; s > 0; s >>= 1) {
155         if (tid < s) {
156             sdata[tid] += sdata[tid + s];  // + instead of *
157         }
158         __syncthreads();
159     }
160 
161     if (tid == 0) {
162         partialSums[blockIdx.x] = sdata[0];
163     }
164 }
165 
166 __global__ void reduceSumFinal(int *partialSums, int *result
    , int size)
167 {
168     extern __shared__ int sdata[];
169 
170     int tid = threadIdx.x;
171     sdata[tid] = (tid < size) ? partialSums[tid] : 0;  //
        identity for sum is 0
172     __syncthreads();
173 
174     for (int s = blockDim.x / 2; s > 0; s >>= 1) {
175         if (tid < s) {
176             sdata[tid] += sdata[tid + s];
177         }
178         __syncthreads();
179     }
180 
181     if (tid == 0) {
182         *result = sdata[0];
183     }
184 }
185 
186 void printMatrix(const std::vector<int>& m, int rows, int
    collumns) {
187     for(int r = 0; r < rows; r++) {
188         for(int c = 0; c < collumns; c++) {
189             std::cout << m[r * collumns + c] << "␣";
190         }
191         std::cout << std::endl;
192     }
193 }
194 
195 void printArray(const std::vector<int>& m, int size) {
196     //     std:: cout << "[";
197     // for(int r = 0; r < size; r++) {
198     //     if(r!=size-1){
199     //         std::cout << m[r] << ", ";
200     //     }
```

```
201     //      else{std::cout << m[r];}
202     // }
203     //          std::cout << "]"<< std::endl;
204 
205 }
206 void printArrayFloat(const std::vector<float>& m, int size)
        {
207     //      std:: cout << "[";
208     // for(int r = 0; r < size; r++) {
209     //      if(r!=size-1){
210     //          std::cout << m[r] << ", ";
211     //      }
212     //      else{std::cout << m[r];}
213     // }
214     //          std::cout << "]"<< std::endl;
215 
216 }
217 void printResult(const std::vector<int>& m1, const std::
        vector<int>& m2, const std::vector<int>& result_matrix,
        int rows, int collumns){
218         std::cout << std::string(20, '=') << std::endl<<std
                ::endl;
219         printMatrix(m1, rows, collumns);
220         std::cout<<std::endl << "␣plus␣min␣maal,␣denk␣na␣en␣
                je␣zult␣het␣wel␣zien␣" << std::endl<<std::endl;
221         printMatrix(m2, rows, collumns);
222         std::cout << std::string(20, '-') << std::endl;
223         printMatrix(result_matrix, rows, collumns);
224         std::cout << std::string(20, '=') << std::endl<<std
                ::endl;
225 }
226 
227 std::vector<int> generateMatrix(int rows, int collumns) {
228     std::vector<int> matrix(rows * collumns);
229     for(int i = 0; i < rows * collumns; i++)
230         matrix[i] = rand() % 100;
231     return matrix;
232 }
233 
234 std::vector<int> generateArray(int size) {
235     std::vector<int> array(size);
236     for(int i = 0; i < size; i++)
237         array[i] = rand() % 10;  //rest van deling door 10
                -> 0-9, veilig voor mult (als size<max_int)
238     return array;
239 }
240 std::vector<float> generateArrayfloat(int size) {
241     std::vector<float> array(size);
242     for(int i = 0; i < size; i++)
```

```
243         array[i] = static_cast<float>(rand()) / static_cast<
                float>(RAND_MAX); // [0.0, 1.0]
244     return array;
245 }
246 int main() {
247     std::vector<int> sizes = {50000, 250000, 500000,
            1000000}; // Test size
248     srand(42);
249     float ms;
250 
251     int output;
252     int threads1D = 256;
253 
254     int* d_a1, *d_a3, *d_a4;
255     float* d_a2;
256     int* d_result_matrix;
257     int* d_out;
258     int *d_partialSum, *d_sumResult;
259     cudaEvent_t start_cuda, stop_cuda;
260     cudaEventCreate(&start_cuda);
261     cudaEventCreate(&stop_cuda);
262 
263     std::cout << "␣sync:␣" << std::endl << std::setw(20) <<
            "size" << "␣" << "␣time"<<std::endl;
264 
265     for(int size : sizes){
266         cudaEventRecord(start_cuda);
267         for(int iterations = 0; iterations < 50; iterations
                ++){
268 
269         //std::cout << std::string(90, '-') << std::endl;
270 
271         int rows = size;
272         int columns = size;
273         int total_elements = size;
274 
275         int blocks1D = (total_elements + threads1D - 1) /
                threads1D;
276         float *d_result;
277         std::vector<int> m1 = generateArray(size);
278         std::vector<float> m2 = generateArrayfloat(size);
279         std::vector<int> m3 = generateArray(size);
280         std::vector<int> m4 = generateArray(size);
281 
282         std::vector<int> result_matrix(total_elements);
283 
284         //std::cout << std::setw(10) << size<<std::endl;
285         // Allocate device memory
286         cudaMalloc(&d_a1, total_elements * sizeof(int));
287         cudaMalloc(&d_a2, total_elements * sizeof(float));
```

```
288         cudaMalloc(&d_a3, total_elements * sizeof(int));
289         cudaMalloc(&d_a4, total_elements * sizeof(int));
290
291         cudaMalloc(&d_result_matrix, total_elements * sizeof
                (int));
292
293         cudaMemcpy(d_a1, m1.data(), total_elements * sizeof(
                int), cudaMemcpyHostToDevice);
294         cudaMemcpy(d_a2, m2.data(), total_elements * sizeof(
                float), cudaMemcpyHostToDevice);
295         cudaMemcpy(d_a3, m3.data(), total_elements * sizeof(
                int), cudaMemcpyHostToDevice);
296         cudaMemcpy(d_a4, m4.data(), total_elements * sizeof(
                int), cudaMemcpyHostToDevice);
297         // Matrix sum
298         cudaMalloc(&d_partialSum, blocks1D * sizeof(float));
299         cudaMalloc(&d_sumResult, sizeof(int));
300
301         reduceSum<<<blocks1D, threads1D, threads1D * sizeof(
                int)>>>(d_a1, d_partialSum, size);
302
303         int finalThreads = 1;
304         while (finalThreads < blocks1D) finalThreads <<= 1;
305         reduceSumFinal<<<1, finalThreads, finalThreads *
                sizeof(int)>>>(d_partialSum, d_sumResult,
                blocks1D);
306
307         int sumOutput;
308         cudaMemcpy(&sumOutput, d_sumResult, sizeof(int),
                cudaMemcpyDeviceToHost);
309         printArray(m1, size);
310         //std::cout << "Total sum: " << sumOutput << std::
                endl;
311
312         cudaFree(d_partialSum);
313         cudaFree(d_sumResult);
314
315         // multiplication
316         float *d_partial;
317         float output_mult;
318         cudaMalloc(&d_partial, blocks1D * sizeof(float));
319         cudaMalloc(&d_result, sizeof(float));
320
321         reduceMultiplyFloat<<<blocks1D, threads1D, threads1D
                *sizeof(float)>>>(d_a2, d_partial, total_elements
                );
322         finalThreads = 1;
323         while (finalThreads < blocks1D) finalThreads <<= 1;
324         reduceMultiplyFinalFloat<<<1, finalThreads,
                finalThreads*sizeof(float)>>>(d_partial, d_result
```

```
        , blocks1D);

    cudaMemcpy(&output_mult, d_result, sizeof(float),
        cudaMemcpyDeviceToHost);
    printArrayFloat(m2, size);

    //std::cout << "total mult: "<<output_mult<<std::
        endl;

    //max
    // Allocate temporary memory for reduction
    int max_size = total_elements;
    cudaMalloc(&d_out, ((max_size + threads1D - 1) /
        threads1D) * sizeof(int));
    int* d_in = d_a3; // Use original matrix for
        reduction
    int* d_tmp = d_out;

    // Max reduction
    int current_n = max_size;
    while(current_n > 1){
        int blocksPerGrid = (current_n + threads1D - 1)
            / threads1D;
        findMaxReductionKernel<<<blocksPerGrid,
            threads1D>>>(d_in, d_tmp, current_n);
        cudaDeviceSynchronize();

        // Swap pointers
        int* temp = d_in;
        d_in = d_tmp;
        d_tmp = temp;

        current_n = blocksPerGrid;
    }
    cudaMemcpy(&output, d_in, sizeof(int),
        cudaMemcpyDeviceToHost);
    printArray(m3, size);

    //std::cout << "Max value: " << output << std::endl;

    // Min reduction (same idea)
    current_n = max_size;
    d_in = d_a4;
    d_tmp = d_out;
    while(current_n > 1){
        int blocksPerGrid = (current_n + threads1D - 1)
            / threads1D;
        findMinReductionKernel<<<blocksPerGrid,
            threads1D>>>(d_in, d_tmp, current_n);
        cudaDeviceSynchronize();
```

```
365
366             int* temp = d_in;
367             d_in = d_tmp;
368             d_tmp = temp;
369
370             current_n = blocksPerGrid;
371         }
372         cudaMemcpy(&output, d_in, sizeof(int),
                cudaMemcpyDeviceToHost);
373         printArray(m4, size);
374
375         //std::cout << "Min value: " << output << std::endl;
376
377         // Free GPU memory
378         cudaFree(d_a1);cudaFree(d_a2);cudaFree(d_a3);
                cudaFree(d_a4);
379         cudaFree(d_result_matrix);
380         cudaFree(d_out);
381         cudaFree(d_partial);  // add this before cudaFree(
                d_matrix1)
382         cudaFree(d_result);   // same for d_result
383     }
384         cudaEventRecord(stop_cuda);
385         cudaEventSynchronize(stop_cuda);
386         cudaEventElapsedTime(&ms, start_cuda, stop_cuda);
387         float avgTime = ms/50;
388         std::cout << std::setw(20) << size << "␣" << avgTime
                <<std::endl;
389     }
390
391     return 0;
392 }
```